ATHANASIOS GOURDOMICHALIS
Numerical Analysis Python Programs
Monte Carlo Integration Between the Graphs of Two Functions

This document provides:

- A short intro to Monte Carlo Integration
- The program's problem statement (assignment prompt and requirements)
- The code description and implementation details.

1. Introduction to Monte Carlo Integration

Monte Carlo Integration is a stochastic **numerical method** used to **approximate definite integrals** and **geometric areas** through **uniform random sampling**. By enclosing an arbitrary target domain $\Omega \subset \mathbb{R}^2$ within a bounding domain **D of known area** A_D, we generate **n** independent uniformly distributed random points $(p_x, p_y) \in D$.

The **indicator condition** determines whether **each point falls inside** Ω:

$$\text{Area}(\Omega) \approx A_D \times \frac{N_{inside}}{N_{total}}$$

- Statistical Averaging & Law of Large Numbers:
  - Because **Monte Carlo approximations** depend on **random sampling**, individual runs exhibit **stochastic variance**.
  - Repeating the simulation over **M independent trials** and **evaluating the arithmetic mean** yields a **more stable and accurate estimate** of the **integral**:

$$\bar{I} = \frac{1}{M}\sum_{j=1}^{M} I_j$$

2. Problem Statement

Given Functions (parabolas):

$$P_1(x) = x^2 - x + \frac{1}{2} \quad \text{and} \quad P_2(x) = -x^2 + x + \frac{1}{2}$$

- Both curves **intersect** at $x = 0$ and $x = 1$, enclosing **a planar domain** in $\mathbb{R}^2$ entirely **contained in the unit square** $[0, 1] \times [0, 1]$.
- **Requirements**:
  1. **Monte Carlo Simulation:** We must estimate the **enclosed area** as the **ratio of random points** falling **between the parabolas inside** $[0, 1] \times [0, 1]$. Each experiment uses $n = 10^3$ **random points, repeated across 10 independent simulation trials**. The final estimate **is the average of all 10 experimental results**.
  2. **Curve & Sample Points Visualization:** Using `matplotlib`, we plot the curves of $P_1(x)$ and $P_2(x)$ together with the **generated sample points** from **the first experiment** $(i = 0)$. Points inside **the region are plotted in** green (`'g<'`) and **points outside** in red (`'r>'`), complete with **axis limits, labels, a title**, and **a legend**.
  3. **Analytical Verification with** `SymPy`: We define the **symbolic expressions** of $P_1(x)$ and $P_2(x)$, we formulate the **area integral** $I = \int_0^1 \big(P_2(x) - P_1(x)\big)\, dx$, we compute **the exact fraction** and **decimal value** using `sp.integrate`, and we **compare it** against the **Monte Carlo estimate**.

3. Code Description & Implementation Details

- **Initial Setup & Declarations:**
  - We import `numpy`, `matplotlib.pyplot`, and `sympy` as `sp`.
  - `np.random.seed(0)` guarantees **numerical reproducibility**.

- `random(a, b)` generates **uniform floating-point coordinates** over $[a, b]$ via $a + (b - a) \times$ `rand()`.
- Both **parabolic boundaries** $P_1(x) = x^2 - x + 1/2$ and $P_2(x) = -x^2 + x + 1/2$ are **defined** as **standard** `Python` **functions**.

- **Simulation Loop:**
  - We iterate **across** `num = 10` **trials** using an **outer loop** (`for i in range(num)`).
  - **In each trial**, counters `tot = cntIn = 0` track **sample points**.
  - For each of the $n = 1000$ `points`, coordinates (p_x, p_y) are generated in $[0, 1] \times [0, 1]$.
  - **Point Inclusion Criterion:** We check if $P_1(p_x) \leq p_y \leq P_2(p_x)$. If true, `cntIn` **increments by** 1. During the first trial ($i = 0$), inside points are **rendered via** `plt.plot(p_x, p_y, 'g<')`, while outside points are **rendered via** `plt.plot(p_x, p_y, 'r>')`.
  - The **trial area** is **appended** to **estimates** via `integral_exp = cntIn / tot` **(since** $A_{\text{square}} = 1$**)**.
  - The **final estimate** is **computed** via `Integral = sum(estimates) / len(estimates)` yielding ≈ 0.3266.

- **Curve Visualization:**
  - A continuous domain $x \in [0, 1]$ is discretized with 300 **points** using `np.linspace(0, 1, 300)`.
  - `plt.plot(x, P1(x), 'k')` and `plt.plot(x, P2(x), 'm')` display P_1 **(black)** and P_2 **(magenta)** above **the scatter points**, framed by $[0, 1] \times [0, 1]$ **axis limits**.

- **Analytical Evaluation via SymPy:**
  - **We declare symbol x** and **define symbolic expressions** with exact **rational coefficients** (`sp.Rational(1, 2)`).
  - `sp.Integral(P2 - P1, (x, 0, 1))` sets up the **exact integral expression** $\int_0^1(-2x^2 + 2x)\ dx$.
  - `sp.integrate(P2 - P1, (x, 0, 1))` evaluates **the exact area** to $1/3$ ($0.333\ldots = 0.\overline{3}$), confirming the **accuracy of the Monte Carlo result** (0.3266).